

Genetic Algorithms for Engine Performance Inverse Problems with Sensor Faults

Technical Report

By

METSA WOWO Glorie & SANAZ HAMIDI

Smart Data Science, ENSAI

January 29, 2026

Abstract

This report describes a comprehensive experimental framework designed to evaluate the performance of a Genetic Algorithm (GA) for estimating gas turbine engine health indicators from sensor measurements. The framework systematically tests the GA across **24 different experimental scenarios**, varying the number of flight contexts (1–4), sensors (1–3), and data quality (clean vs. noisy). The experiments quantify the impact of sensor configuration and measurement noise on estimation accuracy, providing insights for optimal sensor selection in real-world applications.

Contents

1	Introduction	3
2	Problem Statement	3
2.1	The Inverse Problem	3
2.2	Health Indicators	3
3	Experimental Methodology	3
3.1	Genetic Algorithm Configuration	4
3.1.1	Key GA Operators	4
3.2	Experimental Scenarios	4
3.2.1	Flight Contexts (1–4)	4
3.2.2	Sensors (1–3)	5
3.2.3	Scenario Matrix	5
3.3	Noise Model	5
3.3.1	Noise Application Process	5
3.4	Fitness Function	5
3.5	Ablation Study	6
3.5.1	Interpretation	6
4	Simulator Cache Mechanism	6
4.1	Motivation	6
4.2	Cache Principle	6
4.3	Effect on Simulator Calls	7
4.3.1	Without cache	7
4.3.2	With cache	7
4.3.3	Typical behavior in practice	7
4.4	Key takeaway	7
5	Outputs and Visualizations	8
5.1	Per-Scenario Outputs	8
5.2	Summary Outputs	8
5.3	Metrics Tracked	8
6	Key Findings	8
7	Output Directory Structure	9
8	Conclusion	9

1 Introduction

Gas turbine engines are critical components in aviation and power generation systems. Monitoring the health of these engines is essential for ensuring safe operations, reducing maintenance costs, and preventing catastrophic failures. This report presents an experimental framework for evaluating a Genetic Algorithm (GA)-based approach to estimating engine health indicators from sensor measurements.

The framework addresses the fundamental challenge of the *inverse problem*: determining internal component degradation states from external sensor readings. Traditional analytical methods often fail due to the complexity and non-linearity of gas turbine thermodynamic models.

2 Problem Statement

2.1 The Inverse Problem

Gas turbine health monitoring requires estimating internal component degradation from external sensor measurements. This is an **inverse problem** characterized by three fundamental challenges:

1. **Non-invertible equations**: The forward model (degradation $\rightarrow$ measurements) cannot be analytically inverted
2. **Solution multiplicity**: Multiple degradation states can produce similar sensor readings
3. **Sensor uncertainties**: Real measurements contain noise that complicates the estimation

Mathematically, if $\mathbf{y} = f(\boldsymbol{\theta})$ represents the forward model where $\boldsymbol{\theta}$ is the vector of health indicators and $\mathbf{y}$ is the sensor measurement vector, we seek to find:

$$\hat{\boldsymbol{\theta}} = \arg \min_{\boldsymbol{\theta}} \|\mathbf{y}_{\text{measured}} - f(\boldsymbol{\theta})\| \quad (1)$$

The GA provides a global optimization approach to solve this ill-posed inverse problem without requiring gradient information.

2.2 Health Indicators

The framework estimates three key health indicators representing component degradation:

Table 1: Health indicators to be estimated

Indicator	Short Name	Physical Meaning
deg_CmpFan_s_mapWc_in	Fan Wc	Fan mass flow capacity
deg_CmpH_s_mapEff_in	HPC Eff	High-pressure compressor efficiency
deg_TrbH_s_mapEff_in	HPT Eff	High-pressure turbine efficiency

These indicators capture degradation in the three most critical rotating components of a typical turbofan engine.

3 Experimental Methodology

3.1 Genetic Algorithm Configuration

The GA uses an adaptive configuration with parameters optimized through preliminary experiments:

Table 2: GA configuration parameters

Parameter	Value	Purpose
Population Size	150	Balance between diversity and computational cost
Max Generations	250	Allow sufficient convergence time
Tournament Rate	10%	Moderate selection pressure
Elitism Rate	5%	Preserve best solutions
Crossover Rate (BLX- α)	85%	High recombination for exploration
BLX- α Parameter	0.5	Extended exploration beyond parents
Mutation Rate	50%	High mutation for diversity
Initial Mutation Strength	0.15	Broad initial search
Mutation Decay	0.998	Gradual refinement
Min Mutation Strength	0.01	Maintain minimum exploration
Early Stop Generations	45	Efficiency optimization

3.1.1 Key GA Operators

BLX- α Crossover For parents p_1 and p_2 , offspring genes are sampled from:

$$c_i \sim \mathcal{U}[\min(p_{1i}, p_{2i}) - \alpha \cdot d_i, \max(p_{1i}, p_{2i}) + \alpha \cdot d_i] \quad (2)$$

where $d_i = |p_{1i} - p_{2i}|$ and $\alpha = 0.5$.

Adaptive Mutation Mutation strength decays exponentially:

$$\sigma_g = \max(\sigma_0 \cdot \lambda^g, \sigma_{\min}) \quad (3)$$

where $\sigma_0 = 0.15$, $\lambda = 0.998$, and $\sigma_{\min} = 0.01$.

3.2 Experimental Scenarios

The framework tests all combinations of the following configurations:

3.2.1 Flight Contexts (1–4)

- **CRUISE** – Steady-state cruise conditions
- **TAKEOFF** – High-power takeoff phase
- **CLIMB1** – Initial climb segment
- **CLIMB2** – Continued climb segment

Each context provides different operating conditions where component degradation manifests differently in sensor readings.

3.2.2 Sensors (1–3)

- **HPC_Tin** – High-pressure compressor inlet temperature
- **LPT_Tin** – Low-pressure turbine inlet temperature
- **HPC_Pout_st** – High-pressure compressor outlet static pressure

3.2.3 Scenario Matrix

The complete experimental design yields 12 configurations per data type:

Table 3: Scenario matrix: number of measurements per configuration

Contexts	1 Sensor	2 Sensors	3 Sensors
1 (CRUISE)	1 meas.	2 meas.	3 meas.
2 (TAKEOFF)	1 meas.	2 meas.	3 meas.
3 (CLIMB1)	1 meas.	2 meas.	3 meas.
4 (CLIMB2)	1 meas.	2 meas.	3 meas.

With clean and noisy data variants, the total is **24 experimental scenarios**.

3.3 Noise Model

Realistic measurement noise is added using context-specific **multivariate Gaussian distributions**. Each flight context has a 6×6 diagonal covariance matrix Σ_c defining sensor noise characteristics.

3.3.1 Noise Application Process

For each data row i and flight context c :

1. Sample noise vector: $\epsilon_c \sim \mathcal{N}(\mathbf{0}, \Sigma_c)$
2. Add to measurements: $\tilde{\mathbf{y}}_{i,c} = \mathbf{y}_{i,c} + \epsilon_c$

The covariance matrices are provided in `noise_covariances.pkl`, with CRUISE typically exhibiting lower noise than TAKEOFF/CLIMB phases due to more stable operating conditions.

3.4 Fitness Function

The GA minimizes the **Normalized Root Mean Square Error (NRMSE)** between predicted and target sensor measurements:

$$\text{Fitness}(\theta) = \sqrt{\frac{1}{N} \sum_{c \in \mathcal{C}} \sum_{s \in \mathcal{S}} \left(\frac{y_{c,s}^{\text{pred}}(\theta) - y_{c,s}^{\text{target}}}{\sigma_{c,s}} \right)^2} \quad (4)$$

where:

- $N = |\mathcal{C}| \times |\mathcal{S}|$ is the total number of measurements
- $\mathcal{C}$ is the set of flight contexts
- $\mathcal{S}$ is the set of sensors
- $\sigma_{c,s}$ is the normalization factor (standard deviation of sensor s in context c)

3.5 Ablation Study

For scenarios with multiple sensors or contexts, an **ablation study** quantifies component importance:

1. **Baseline:** Run GA with all sensors/contexts, record $\text{RMSE}_{\text{base}}$
2. **Sensor ablation:** For each sensor s , run GA without s , record RMSE_{-s}
3. **Context ablation:** For each context c , run GA without c , record RMSE_{-c}

The **impact** of removing component x is computed as:

$$\text{Impact}_x = \frac{\text{RMSE}_{-x} - \text{RMSE}_{\text{base}}}{\text{RMSE}_{\text{base}}} \times 100\% \quad (5)$$

3.5.1 Interpretation

Table 4: Ablation result interpretation

Impact Range	Classification	Meaning
$> +20\%$	High Importance	Critical for accuracy
$+5\%$ to $+20\%$	Moderate Importance	Contributes meaningfully
-5% to $+5\%$	Neutral	Redundant information
$< -5\%$	Detrimental	Adds noise, removal helps

4 Simulator Cache Mechanism

4.1 Motivation

Evaluating the fitness of an individual requires running a physical simulator (`decksmr_1forall`), which is computationally expensive. During a Genetic Algorithm (GA) run, the same individuals are often evaluated multiple times due to elitism, population convergence, and duplicate offspring.

4.2 Cache Principle

To avoid redundant simulator calls, a cache mechanism is introduced. Simulator outputs are stored and indexed by the health state vector:

```
key = tuple(np.round(individual, 10))
if key not in cache:
    cache[key] = run_simulator(individual)
```

Once a health state has been evaluated, its simulator output is reused whenever the same state reappears. As a result, repeated evaluations do not increase the number of simulator calls.

The total number of simulator executions therefore depends on the number of *unique* health states explored, rather than on the total number of fitness evaluations.

4.3 Effect on Simulator Calls

Let:

- P be the population size,
- G be the number of generations,
- U be the number of **unique** individuals evaluated.

4.3.1 Without cache

Without caching, every fitness evaluation triggers a simulator execution:

$$\text{Calls}_{\text{no cache}} = P \times G$$

Example For $P = 80$ and $G = 50$:

$$\text{Calls}_{\text{no cache}} = 80 \times 50 = 4000$$

4.3.2 With cache

With caching enabled, the simulator is executed only once per unique individual:

$$\text{Calls}_{\text{cache}} = U \quad \text{with } U \leq P \times G$$

Duplicate individuals arising from elitism, convergence, or identical offspring do not trigger additional simulator runs.

4.3.3 Typical behavior in practice

As the GA converges, population diversity decreases and many individuals are re-evaluations of the same health states. Consequently:

$$U \ll P \times G$$

Realistic example For $P = 80$ and $G = 50$, a typical run explores approximately $U \approx 800$ unique individuals:

$$\text{Calls}_{\text{cache}} \approx 800 \quad \text{instead of } 4000$$

4.4 Key takeaway

The cache mechanism reduces the computational complexity of the GA from:

$$\mathcal{O}(P \times G) \rightarrow \mathcal{O}(U)$$

This leads to a significant reduction in computational cost once the population starts converging, making large-scale GA-based inverse problems tractable.

5 Outputs and Visualizations

5.1 Per-Scenario Outputs

Each of the 24 scenarios generates the following files:

Table 5: Per-scenario output files

File	Description
results.csv	Per-row estimation results (true values, estimates, errors, timing)
results_summary.png	4-panel plot: RMSE distribution, fitness, per-indicator errors, true vs. estimated scatter
convergence.png	6-panel convergence analysis: fitness evolution, mutation decay, diversity, indicator convergence, error convergence, final errors
trajectory_3d.png	3D visualization of GA search path in indicator space
ablation.csv	Ablation study results (if applicable)
ablation_plot.png	4-panel ablation visualization
multiple_solutions_plot.png	All solution obtained to estimate a measurement

5.2 Summary Outputs

Table 6: Summary output files

File	Description
experiment_summary.csv	Aggregated statistics for all 24 scenarios
summary_comparison.png	Clean vs. noisy performance comparison across all configurations
execution_time_analysis.png	Computational cost analysis: time vs. measurements, time vs. simulator calls

5.3 Metrics Tracked

Table 7: Performance metrics

Metric	Description
mean_rmse, std_rmse	Estimation accuracy (lower is better)
mean_fitness, std_fitness	GA convergence quality
mean_time, std_time	Execution time per estimation (seconds)
total_time	Total scenario execution time
mean_sim_calls	Average simulator calls per estimation
mean_generations	Average generations until convergence

6 Key Findings

Based on the experimental design, we anticipate the following results:

1. **More measurements improve accuracy:** Configurations with more contexts and sensors achieve lower RMSE, as additional information constrains the solution space.
2. **Noise degrades performance:** Noisy data should consistently show higher RMSE than clean data, with the degradation proportional to noise variance, even though the result show the contrary in some cases.
3. **Sensor importance varies:** Ablation studies reveal which sensors contribute the most to estimation accuracy, some sensors may be redundant or even detrimental.
4. **Computational cost scales:** More measurements require more simulator calls and execution time.

7 Output Directory Structure

The experiment generates a structured output directory:

```
experiments/run_YYYYMMDD_HHMMSS/
|-- experiment_summary.csv
|-- summary_comparison.png
|-- execution_time_analysis.png
|-- clean/
|   |-- 1ctx_1sensors/
|   |   |-- results.csv
|   |   |-- results_summary.png
|   |   |-- convergence.png
|   |   +-- trajectory_3d.png
|   |   +-- multiple_solutions.csv
|   |-- 1ctx_2sensors/
|   |   |-- results.csv
|   |   |-- ... (+ ablation.csv, ablation_plot.png)
|   +-- ... (12 scenarios total)
+-- noisy/
    +-- ... (same structure as clean/)
```

8 Conclusion

This experimental framework provides a rigorous methodology for evaluating GA-based health indicator estimation under varying conditions. By systematically testing 24 scenarios and generating comprehensive visualizations, the framework enables:

- **Optimal sensor selection** for deployment, identifying which sensors provide the most informative measurements
- **Quantification of noise impact** on estimation accuracy, establishing performance bounds for real-world applications
- **Understanding of computational trade-offs** between accuracy and speed, informing real-time implementation decisions

The results can directly inform industrial decisions about sensor placement and redundancy requirements for gas turbine health monitoring systems. Future work may extend this framework to include additional health indicators, alternative optimization algorithms, and time-series trajectory estimation.